



JPA - CascadeType y orphanRemoval

Simplificando la persistencia de entidades relacionadas

Una guía práctica para automatizar la gestión de relaciones entre entidades en JPA, eliminando la complejidad del manejo manual de persistencia.

El problema: persistencia manual

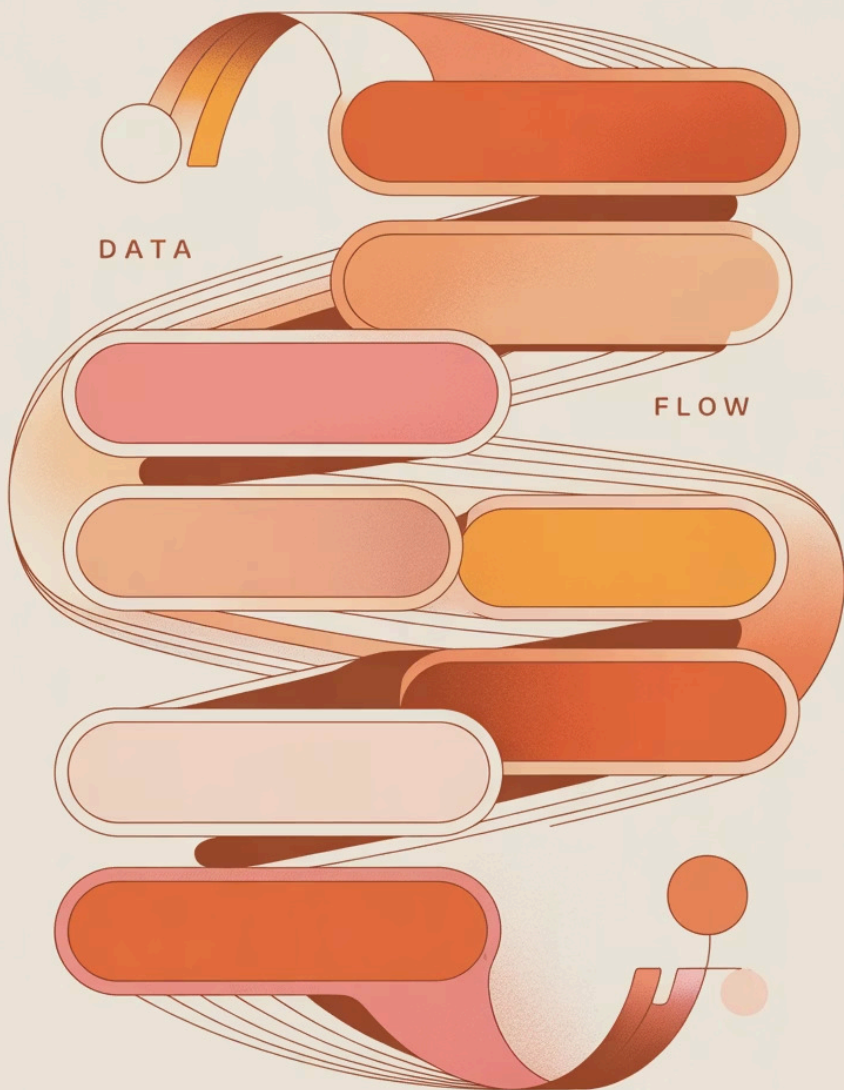
En nuestros ejemplos anteriores, hemos visto cómo manejar relaciones OneToOne requiere persistir cada entidad por separado. Este enfoque funciona correctamente, pero puede volverse tedioso y propenso a errores cuando trabajamos con múltiples relaciones.

```
// Código del ejemplo anterior - persistencia manual
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();

tx.begin();
// Primero persistimos el domicilio
em.persist(domicilio);
// Luego persistimos la persona
em.persist(persona);
tx.commit();
```

Como podemos observar, necesitamos recordar el orden correcto de persistencia y manejar cada entidad individualmente.





¿Qué es Cascade en JPA?

Definición

Cascade es un mecanismo que **propaga automáticamente** las operaciones realizadas en una entidad principal hacia sus entidades relacionadas.

Beneficios

Elimina la necesidad de manejar manualmente la persistencia de entidades relacionadas, **simplificando significativamente** nuestro código.

Cuando aplicamos cascade a una relación Persona → Domicilio, las operaciones sobre Persona se propagan automáticamente a Domicilio, manteniendo la consistencia de datos sin intervención manual.

Tipos de CascadeType disponibles



PERSIST

Propaga la operación de **persistencia**. Al guardar la entidad principal, también se guardan las relacionadas.

MERGE

Propaga las **actualizaciones**. Los cambios en la entidad principal se reflejan en las relacionadas.



REMOVE

Propaga el **borrado**. Eliminar la entidad principal también elimina las relacionadas.

REFRESH

Propaga la **recarga** de datos desde la base de datos hacia las entidades relacionadas.

DETACH

Propaga la **desconexión** del contexto de persistencia hacia las entidades relacionadas.

ALL

Incluye **todas las operaciones** anteriores. Es equivalente a aplicar todos los tipos de cascade.

 Los tipos más utilizados en el desarrollo cotidiano son **PERSIST**, **MERGE** y **REMOVE**, ya que cubren las operaciones básicas de CRUD.

Ejemplo práctico: Persona y Domicilio

Implementación con Cascade

```
@Entity
public class Persona {
    @Id
    @GeneratedValue
    private Long id;

    private String nombre;

    @OneToOne(cascade = CascadeType.PERSIST)
    @JoinColumn(name = "domicilio_id")
    private Domicilio domicilio;

    // getters y setters
}
```

Código simplificado en Main

```
// Ahora solo necesitamos persistir Persona
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();

tx.begin();
em.persist(persona); // ¡Domicilio se guarda automáticamente!
tx.commit();
```



✓ **Ventaja clave:** Al persistir la Persona, JPA automáticamente detecta la relación y persiste también el Domicilio asociado, eliminando la necesidad de llamadas manuales a `em.persist(domicilio)`.

Ejemplo avanzado: Persona y Curso (ManyToMany)



Entidad Persona

```
@Entity
public class Persona {
    @Id
    @GeneratedValue
    private Long id;

    private String nombre;

    @ManyToMany(cascade = CascadeType.PERSIST)
    @JoinTable(
        name = "persona_curso",
        joinColumns = @JoinColumn(name = "persona_id"),
        inverseJoinColumns = @JoinColumn(name = "curso_id")
    )
    private List<Curso> cursos = new ArrayList<>();
}
```



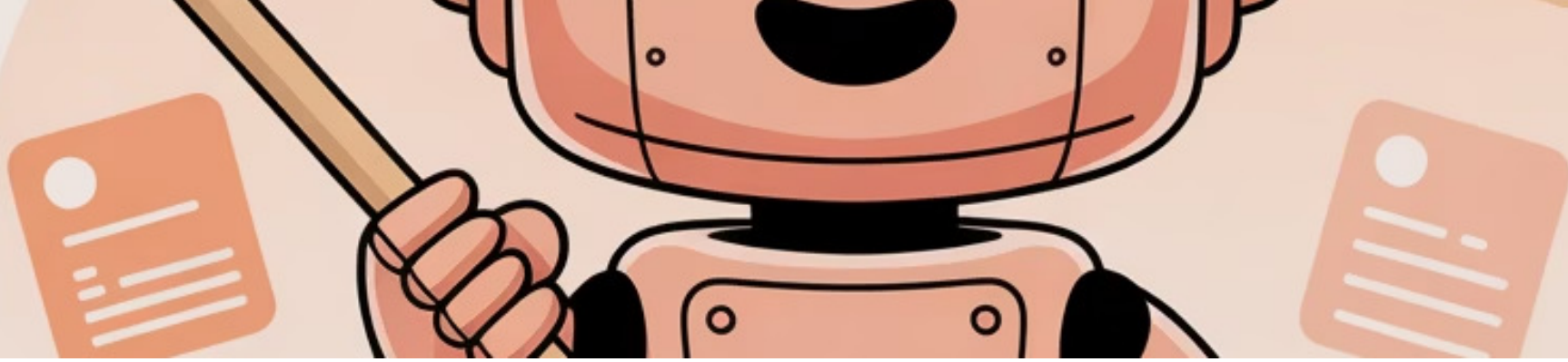
Resultado en código

```
// Crear persona y cursos
Persona persona = new Persona("Juan Pérez");
Curso java = new Curso("Java Avanzado");
Curso spring = new Curso("Spring Framework");

persona.getCursos().add(java);
persona.getCursos().add(spring);

// Solo persistir persona
em.persist(persona);
// ¡Los cursos también se guardan automáticamente!
```

Con **cascade = CascadeType.PERSIST**, al guardar una Persona, todos los cursos asociados se persisten automáticamente en la base de datos, manteniendo la integridad referencial.



orphanRemoval: eliminación automática de huérfanos

¿Qué son las entidades huérfanas?

Son entidades que **pierden su relación** con la entidad principal. Por ejemplo, cuando una Persona ya no tiene asociado un Domicilio, ese Domicilio se convierte en "huérfano".

Configuración

```
@Entity
public class Persona {
    @OneToOne(cascade = CascadeType.PERSIST,
              orphanRemoval = true)
    @JoinColumn(name = "domicilio_id")
    private Domicilio domicilio;
}
```

Comportamiento automático

```
// Si modificamos la relación
persona.setDomicilio(null);
em.merge(persona);

// El domicilio anterior se elimina
// automáticamente de la base de datos
// ¡Sin necesidad de DELETE manual!
```

⚠ **Importante:** orphanRemoval mantiene la base de datos limpia eliminando automáticamente registros que ya no tienen una relación válida, evitando datos residuales.

Buenas prácticas y consideraciones importantes

Cuidado con CascadeType.REMOVE

Puede eliminar datos no deseados en cascada. Úsalo solo cuando estés **completamente seguro** de que las entidades relacionadas deben eliminarse junto con la principal.

orphanRemoval inteligente

Aplica `orphanRemoval = true` únicamente cuando la entidad dependiente **no tenga sentido por sí sola**. Por ejemplo: Domicilio sin Persona.

Cascade selectivo

En lugar de `CascadeType.ALL`, especifica solo los tipos necesarios: `CascadeType.PERSIST`, `CascadeType.MERGE` según tu caso de uso.

Pruebas exhaustivas

Siempre prueba el comportamiento de cascade y orphanRemoval en un **entorno de desarrollo** antes de implementar en producción.



DEMO EN VIVO

Cascade y orphanRemoval en acción

01

Persistencia con Cascade

Mostraremos cómo persistir `Persona` y `Domicilio` usando `CascadeType.PERSIST`, observando cómo se simplifica el código.

02

Relación ManyToMany

Mostraremos la persistencia de `Persona` con múltiples `Cursos`, viendo cómo `cascade` maneja automáticamente las relaciones complejas.

03

orphanRemoval en acción

Eliminaremos la relación entre `Persona` y `Domicilio` para observar cómo `orphanRemoval = true` limpia automáticamente la base de datos.

- 📄 El código completo se ejecutará en vivo para mostrar el comportamiento real de estas funcionalidades en tiempo de ejecución.

Resumen y próximos pasos

Conceptos clave aprendidos



CascadeType

Propaga automáticamente operaciones de persistencia, actualización y eliminación entre entidades relacionadas.



orphanRemoval

Elimina automáticamente entidades huérfanas, manteniendo la integridad y limpieza de la base de datos.



Ejercicio propuesto para practicar

- ❓ **Reto:** Implementa una relación Autor-Libro con cascade apropiado. Considera: ¿Debe un libro existir sin autor? ¿Un autor sin libros? Experimenta con diferentes tipos de cascade y orphanRemoval.

¡Continúa experimentando con estos parámetros para dominar la gestión de relaciones en JPA!